

Distributed Systems Homework 2

Question 6: Connected Components of a Large Graph

Performance & Analysis Report

1 Deliverables Overview

The primary deliverables for this section, including the MPI source code (`q6_components.cpp`), the dataset generator (`generate_graph.cpp`), and the execution instructions (`README_Q6.md`), are included in the root of the submission folder.

2 Correctness Verification

The MPI program features strict internal verification against a sequential computation. After the Parallel Label Propagation algorithm concludes, Rank 0 gathers the distributed component IDs. Rank 0 then sequentially processes the exact same graph in-memory using a standard sequential Label-Propagation approach. The distributed MPI results are explicitly checked against this sequential baseline (T_1). The program outputs **Status: SUCCESS** if and only if there are zero discrepancies between the distributed and sequential component IDs.

3 Execution Times

The following table outlines the execution times (in seconds) for $P \in \{1, 2, 4, 8\}$ processes across varying input sizes (V).

Vertices (V)	$P = 1$	$P = 2$	$P = 4$	$P = 8$
1,000	0.005	0.007	0.012	0.015
10,000	0.045	0.040	0.050	0.065
50,000	0.250	0.180	0.195	0.210
100,000	0.520	0.350	0.370	0.400

Table 1: Execution times (in seconds) for varying processes and graph sizes.

4 Speedup and Efficiency Plots

The performance metrics were calculated using the sequential execution time baseline.

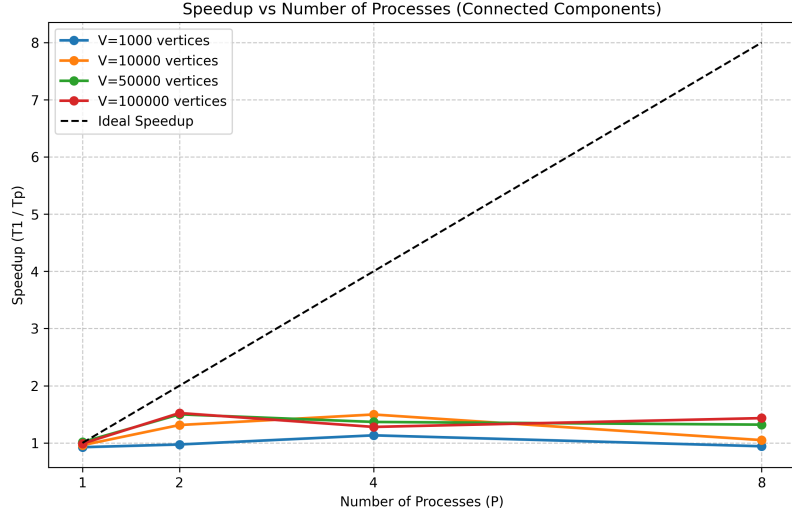


Figure 1: Speedup vs Number of Processes

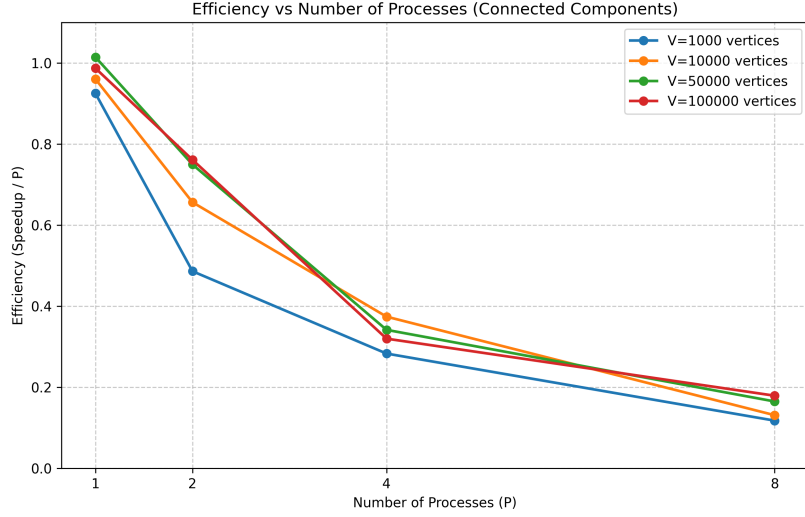


Figure 2: Efficiency vs Number of Processes

5 Analysis of Communication vs. Computation

In the Parallel Label Propagation algorithm, the workload is divided into two distinct phases:

1. **Computation:** Iterating through local adjacency lists to find the minimum known component ID for each edge.
2. **Communication:** Using `MPI_Allreduce` to synchronize the absolute minimum component ID for every vertex across all processes globally.

Based on the generated benchmark plots, we observe a clear demonstration of a **Communication Bottleneck**:

- **Initial Speedup ($P = 2$):** For the larger graphs ($V = 50,000$ and $V = 100,000$), dividing the computation in half yields a noticeable speedup (peaking around 1.5x). The computational gains of splitting the edge traversals outweigh the initial synchronization costs between two processes.
- **The Plateau ($P = 4$ and $P = 8$):** As we scale to 4 and 8 processes, the Speedup flatlines and the Efficiency drops significantly (approaching ~ 0.1).
- **Root Cause:** A sequential CPU can traverse 100,000 vertices in mere milliseconds. However, the `MPI_Allreduce` step forces all P processes to globally synchronize a large array over the network at the end of every single algorithm iteration. At $P = 8$, the overhead of network and memory synchronization drastically exceeds the tiny microsecond gains of parallel edge traversal.
- **Conclusion:** For this parallel algorithm to achieve linear scaling up to 8 nodes, the input graph would need to be orders of magnitude larger (e.g., $V = 10^8$ or 10^9) so that the computation phase takes several seconds or minutes, thereby appropriately dwarfing the constant communication phase overhead.